

Program Layout & Execution

Recursive Function Call Demonstration

CMPE 220 - Spring 2026

San Jose State University

Group 13

Muhammad Tokhi

Miles Nguyen

Myles Easter

Project Report

GitHub Repository

Report / Demonstration Repository:

<https://github.com/muhammادتokhi-sjsu/cmpe220-program-layout-and-execution>

Associated Repository:

<https://github.com/0x-000000/cmpe-220.git>

The associated repository contains the Software CPU project and the added recursive function demonstration for this assignment. The project includes the assembler, emulator, CPU, bus, BIOS, terminal device, ISA files, example programs, and build configuration.

Directory/File	Purpose
src/	C++ source files for the assembler, encoder, CPU, bus, BIOS, terminal, emulator, and command-line entry point.
examples/factorial.c	Simple C driver program with a recursive factorial function.
examples/factorial.fasm	Software CPU assembly version used by the emulator to demonstrate function calls and recursion.
docs/	Report, diagrams, and project notes.
CMakeLists.txt	CMake build configuration for the virtual machine.
README.md	Repository overview, build instructions, run commands, and video link.

Video Demonstration - Program Layout & Execution (Factorial)

<https://youtu.be/7rK0Fj6bZWw>

Environment setup

The toolchain assumes a POSIX shell. Pick the section matching your host OS.

Windows (WSL recommended)

WSL is the preferred path: it gives you the same shell, package manager, and compiler the project is developed against, so the build & test commands below work as written.

Native Windows builds with MSVC also work; you just have to install MSVC and CMake yourself. Pick one of the two options.

Option 1: WSL

- Microsoft's WSL install guide: <https://learn.microsoft.com/en-us/windows/wsl/install>
- Installing a specific distro (Ubuntu recommended):
<https://learn.microsoft.com/en-us/windows/wsl/install#change-the-default-linux-distribution-on-installed>
- Setting up your Linux user account inside WSL:
<https://learn.microsoft.com/en-us/windows/wsl/setup/environment>

Quick start:

```
wsl --install -d Ubuntu
```

After it reboots and you've created your Linux username/password, open the "Ubuntu" app and continue with the Linux steps below.

Option 2: Native Windows (MSVC)

1. Install Visual Studio 2022 with the *Desktop development with C++* workload (gives you MSVC 19.30+): <https://visualstudio.microsoft.com/downloads/>
2. Install CMake (3.29+): <https://cmake.org/download/>

3. Open *x64 Native Tools Command Prompt for VS 2022* (or any PowerShell with MSVC on PATH) in the project directory.

Linux

Install a C++20 toolchain and a recent CMake from your package manager.

Debian / Ubuntu / WSL Ubuntu:

```
sudo apt update  
sudo apt install -y cmake build-essential
```

Fedora / RHEL / Rocky:

```
sudo dnf install -y cmake gcc-c++ make
```

Arch / Manjaro:

```
sudo pacman -S --needed cmake base-devel
```

If your distro ships an older CMake (cmake --version < 3.29), grab a current release from the official download page: <https://cmake.org/download/>.

macOS

Install [Homebrew](#) if you don't already have it:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then install CMake:

```
brew install cmake
```

Download, Compile, and Run

Requirements:

- CMake 3.29+
- C++20 compiler (Clang 14+, GCC 11+, MSVC 19.30+)

Build steps:

Setup:

```
cmake -B build -S .
```

Build:

```
cmake --build build --config Release -j
```

Clean:

```
cmake --build build --target clean
```

Test:

```
ctest --test-dir build -C Release -j0 --output-on-failure
```

Run:

- Assemble and execute the program in the emulator (default action):

```
./build/vm -s path/to/file.fasm
```

- Dump the parsed instruction stream:

```
./build/vm -s path/to/file.fasm -S
```

- Dump encoded bytecode (one halfword per line):

```
./build/vm -s path/to/file.fasm -c
```

- Load a raw 64K binary memory image (no assembly step). May be combined with -s, in which case the image overlays the assembled program byte-for-byte:

```
./build/vm -i path/to/image.bin  
./build/vm -s path/to/file.fasm -i path/to/image.bin
```

- Dump the full 64K memory contents to a core file in the current directory after the CPU halts:

```
./build/vm -s path/to/file.fasm -d
```

Flags

Flag	Purpose
-s <file>	Source .fasm file: assemble and load into userland.
-i <file>	Raw memory image (max 64K): overlays anywhere in the address space.
-S	Dump parsed assembly and exit (requires -s).
-c	Dump encoded bytecode and exit (requires -s).
-d	After halt, dump the full 64K of memory (hex+ASCII) to ./core.

When both -s and -i are supplied the source is loaded first, then the image is written on top.

Wherever the image and the assembled program touch the same address, the image's byte takes precedence.

Memory Dump / Import / Conversion

Dump memory contents after execution to ./core

```
./build/vm -s path/to/file.fasm -d
```

Run python script to convert from dumped memory format to 64KB .bin format

```
python3 dump_to_bin.py core image.bin
```

Run the Virtual Machine by populating the memory file with a .bin file

```
./build/vm -i image.bin
```

Project Overview

Goal:

The goal of this assignment is to show how a simple recursive C-style function can be laid out in memory and executed using the Software CPU design from the earlier project.

For this report, the example program calculates factorial using recursion:

```
factorial(n) = n * factorial(n - 1)
factorial(0) = 1
factorial(1) = 1
```

The driver program calls the recursive function with $n = 5$. The final result is 120.

The project demonstrates three main ideas:

1. Program layout: the executable instructions are loaded into user RAM starting at address 0x0000.
2. Function calls: branch-with-link saves a return address, and ret restores the next instruction address from the stack.
3. Recursion: each recursive call creates a stack frame that stores its own return address and saved argument value.

C Driver Program

The C version shows the high-level program logic. The main function sets an input value, calls factorial, and prints the result.

```
#include <stdint.h>
#include <stdio.h>

uint16_t factorial(uint16_t n) {
    if (n <= 1) {
        return 1;
    }

    return n * factorial(n - 1);
}

int main(void) {
    uint16_t n = 5;
    uint16_t result = factorial(n);

    printf("factorial(%u) = %u\n", n, result);
    return 0;
}
```

The Software CPU does not directly execute this C source. Instead, the same logic is represented in our assembly language so the emulator can assemble it into 16-bit instructions and run it on the custom CPU.

Software CPU Assembly Version

The assembly version uses register r0 for the input argument and also for the return value. The function saves the link register on the stack so recursive calls can return correctly.

```
_start:
    movl r0, #5
    movh r0, #0
    bl fact
    bl print_num

    movl r0, #0x0A
    movl r1, #0x02
    movh r1, #0x80
    str r0, r1
    halt

fact:
    push rbl
    movl r1, #1
    movh r1, #0
    cmp r0, r1
    ble fact_base

    push r0
    sub r0, r0, r1
    bl fact

    pop r1
    mul r0, r1, r0
    ret

fact_base:
    movl r0, #1
    movh r0, #0
    ret

print_num:
    push rbl
    movl r1, #10
    movh r1, #0
    div r2, r0, r1
    mul r3, r2, r1
    sub r3, r0, r3
    push r3

    movl r1, #0
    cmp r2, r1
    ble print_digit

    mov r0, r2
    bl print_num

print_digit:
    pop r0
    movl r1, #0x30
    movh r1, #0
    add r0, r0, r1
    movl r1, #0x02
    movh r1, #0x80
    str r0, r1
    ret
```

Executable Memory Layout

Our Software CPU uses one flat 16-bit address space, giving 64 KB of total addressable memory.

Region	Address Range	Purpose
User RAM	0x0000-0x7FFE	User program, data, and stack.
Terminal MMIO	0x8000-0x800F	Terminal RX, TX, and status registers.
BIOS ROM / IVT	0xFF00-0xFFFF	Interrupt vector table and boot/panic code.

The factorial executable is loaded into user RAM starting at address 0x0000. The stack starts at 0x7FFE and grows downward as functions call each other.

Address Range	Program Section	Description
0x0000-0x0010	_start / driver	Loads 5, calls fact, calls print_num, prints newline, and halts.
0x0012-0x002C	fact function	Recursive factorial logic.
0x002E-0x0054	print_num function	Converts the result to decimal text and prints it.
0x7FFE downward	Stack	Stores return addresses, saved arguments, and temporary values.
0x8002	Terminal TX	Characters written here are printed by the emulator.

The exact instruction offsets are assigned by the assembler. The table above shows the conceptual layout used for explaining program execution.

Function Calls

Function calls are handled with the `bl` and `ret` instructions.

bl fact: branches to the `fact` function and stores the return address in `rbl`.

push rbl: saves the return address on the stack so the function can safely make another call.

ret: pops the saved return address from the stack and loads it into `rip`.

This is necessary for recursion because each recursive call overwrites `rbl`. Saving `rbl` on the stack lets every call return to the correct place.

Example call flow for `factorial(5)`:

```
_start calls fact(5)
fact(5) calls fact(4)
fact(4) calls fact(3)
fact(3) calls fact(2)
fact(2) calls fact(1)
fact(1) reaches the base case and returns 1
```

After the base case returns, the stack unwinds and each previous call multiplies its saved `n` value by the returned result.

```
fact(2) = 2 * 1 = 2
fact(3) = 3 * 2 = 6
fact(4) = 4 * 6 = 24
fact(5) = 5 * 24 = 120
```

Recursion and Stack Usage

Each recursive call creates a small stack frame. In this program, each non-base call saves the return address and the current value of *n*.

Stack Address	Stored Value	Purpose
0x7FFC	return to _start	Saved by fact(5).
0x7FFA	5	Saved argument for fact(5).
0x7FF8	return to fact(5)	Saved by fact(4).
0x7FF6	4	Saved argument for fact(4).
0x7FF4	return to fact(4)	Saved by fact(3).
0x7FF2	3	Saved argument for fact(3).
0x7FF0	return to fact(3)	Saved by fact(2).
0x7FEE	2	Saved argument for fact(2).
0x7FEC	return to fact(2)	Saved by fact(1).

The lowest address shown is the deepest point of the recursion. When the base case returns, values are popped in reverse order, which restores the previous call state.

CPU Execution Flow

The CPU executes the assembled program one instruction at a time using a fetch, decode, execute, and writeback-style cycle.

Stage	What Happens
Fetch	The CPU reads a 16-bit instruction from memory at the address stored in rip.
Decode	The top two instruction bits select ALU, memory, branch, or miscellaneous/immediate behavior.
Execute	The CPU performs the operation, such as sub, mul, cmp, push, pop, bl, or ret.
Writeback	The result is written to a register, memory, or rip. Normal instructions advance rip by 2 bytes.

Important instructions used by the factorial program:

cmp r0, r1: updates the NZCV flags in rfl.

ble fact_base: branches to the base case when n is less than or equal to 1.

sub r0, r0, r1: computes n - 1 before the recursive call.

mul r0, r1, r0: multiplies the saved n value by the recursive result.

Memory-Mapped Output

The program prints the result through the terminal memory-mapped I/O device. The terminal transmit register is located at address 0x8002.

```
movl r1, #0x02
movh r1, #0x80
str  r0, r1
```

After the two move instructions, r1 contains 0x8002. When the CPU executes str r0, r1, the bus recognizes that the address belongs to the terminal device instead of normal RAM. The terminal yields a transmit event, and the emulator prints the character to the host console.

Video Demonstration Summary

The demo video should show the following items:

1. The C recursive factorial function and main driver program.
2. The equivalent Software CPU assembly source.
3. The build and run commands.
4. The memory layout of the executable, stack, terminal MMIO, and BIOS.
5. How bl, push, pop, and ret handle function calls.
6. How recursion creates stack frames, reaches the base case, and unwinds to produce 120.

Team Member Contributions

Muhammad Tokhi

- Created the report structure and final document formatting.
- Wrote the high-level explanation of factorial recursion.
- Documented the memory layout, stack layout, and function call flow.
- Helped prepare the video explanation and walkthrough.

Miles Nguyen

- Integrated the recursive factorial example with the existing Software CPU project.
- Verified instruction behavior for bl, ret, push, pop, arithmetic, and branch instructions.
- Helped document how the executable is loaded and executed through the CPU, bus, RAM, BIOS, and terminal device.
- Assisted with repository organization and build instructions.

Myles Easter

- Tested the factorial program in the emulator.
- Verified that recursive execution produced the expected output.
- Helped explain how the emulator executes the assembled bytecode.
- Assisted with the video demonstration and runtime walkthrough.

Conclusion

This project demonstrates how a recursive C-style function can be represented and executed using the custom Software CPU. The factorial program shows function calls, return addresses, stack frames, recursion, base cases, stack unwinding, arithmetic operations, and memory-mapped terminal output. The final program computes $5!$ and prints 120.